

Agentic-OS

Luis Vergara

Las entidades representadas en mi proyecto son: las ventanas gráficas, el proceso Launcher y el servidor IALearner. Debido a que la estación de trabajo del usuario y el servidor de clasificación son entidades físicamente separadas, consideré que la mejor solución debería fundamentarse en sockets TCP para el intercambio de datos entre ambos, reservando las señales POSIX únicamente para el control local entre el Launcher y sus procesos hijos. El esquema de sincronización a emplear debe evitar que una palabra se pierda o se cuente dos veces, y debe permitir que el hilo clasificador reaccione de forma asíncrona cuando cambia el número de ventanas activas.

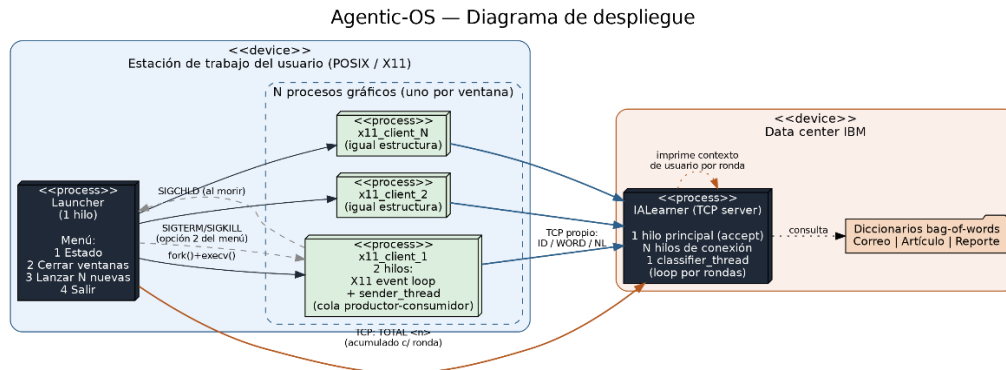


Ilustración 1 Diagrama de solución

El primer proceso es el Launcher y ejecuta las siguientes tareas al iniciarse:

1. Valida los parámetros de entrada y define el manejador de la señal SIGCHLD.
2. Crea la tabla de procesos administrados: un arreglo con el PID, el identificador de ventana y el estado de cada proceso gráfico.
3. Notifica al IALearner, mediante una conexión TCP corta, el número total de ventanas que debe esperar.
4. Crea cada proceso gráfico (x11_client) mediante fork() y execv(), pasándole el identificador de ventana, el host y el puerto como argumentos.
5. Entra a un menú interactivo desde el cual puede consultar el estado de los procesos, cerrar todas las ventanas activas, o lanzar N ventanas adicionales sin reiniciar el sistema, reenviando en ese caso un nuevo total acumulado al IALearner.

El proceso x11_client, uno por cada ventana:

1. Se conecta por TCP al IALearner y se identifica con su número de ventana.
2. Crea un hilo dedicado al envío de datos por red, desacoplado del hilo principal que atiende los eventos de la ventana gráfica, mediante una cola productor-consumidor.
3. Por cada palabra que el usuario termina de escribir (al presionar espacio) la encola para ser enviada; al presionar Enter encola una marca de fin de oración.
4. Al cerrarse la ventana, cierra ordenadamente el hilo de envío y la conexión TCP antes de liberar los recursos gráficos.

El proceso IALearner:

1. Crea la tabla de documentos compartida, protegida por mutex, y lanza el hilo clasificador, que permanece activo durante toda la ejecución del servidor.

Agentic-OS

Luis Vergara

2. Por cada conexión entrante crea un hilo dedicado que interpreta el protocolo de mensajes: identificación de ventana, palabra, fin de oración, o aviso de total de ventanas.
3. Cada hilo de conexión acumula las palabras recibidas en un vector de frecuencias (bag-of-words) propio de esa ventana.
4. Cuando una ventana cierra su conexión, el hilo correspondiente clasifica su documento comparándolo contra los tres diccionarios de referencia y marca el documento como terminado.
5. El hilo clasificador espera hasta que el número de documentos terminados alcanza el total anunciado por el Launcher; en ese momento agrega los resultados de todos los documentos y determina el tipo de usuario.

Con respecto a los tipos de datos, cada ventana tiene su propio mutex y su propio vector de frecuencias, lo cual permite que varias ventanas se procesen en paralelo sin bloquearse entre sí. La tabla de documentos completa está protegida por un mutex adicional junto con una variable de condición que se dispara cada vez que un documento termina o cuando cambia el total de ventanas esperado. La cola de mensajes de cada `x11_client` fue implementada como un buffer circular de tamaño fijo con dos variables de condición (cola llena y cola vacía), siguiendo el patrón productor-consumidor clásico.

Los esquemas de funcionamiento implementados son los siguientes:

1. Envío incremental por palabra: en lugar de enviar la oración completa al terminarla, cada `x11_client` envía cada palabra apenas el usuario la termina de escribir, lo que permite que el `IA Learner` arme el vector de frecuencias sin esperar a que termine toda la oración.
2. Clasificación por rondas acumulativas.- el campo `TOTAL` es un contador que se actualiza: cada vez que el Launcher lanza nuevas ventanas, reenvía un total acumulado (ventanas anteriores más nuevas), y el hilo clasificador reacciona a cada incremento reevaluando el conjunto completo de documentos recolectados hasta ese momento.

Limitaciones del proyecto: si el `IA Learner` no está en ejecución (o pierde la conexión) en el momento en que el Launcher intenta notificar el total de ventanas, el hilo clasificador nunca recibe un valor válido y esa ronda no se clasifica; los procesos `x11_client`, sin embargo, detectan el rechazo de conexión y continúan funcionando en un modo local sin bloquear al usuario.

Para compilar el proyecto debe abrir una terminal dentro de cada carpeta y ejecutar:

- `gcc launcher.c -o launcher -Wall -Wextra -I../include` (Launcher)
- `gcc x11_client.c -o x11_client -lX11 -pthread -Wall -Wextra -I../include` (cliente gráfico),
- `gcc ia_learner.c -o ia_learner -pthread -Wall -Wextra -I../include` (servidor).

Para probarlo, ejecute primero `./ia_learner 9000` en una terminal, y luego `./launcher 3 127.0.0.1 9000` en otra, donde el primer argumento es el número de ventanas a crear.